# GoWrench Inc. Autonomous Driving Vehicle Project Final Report

SEP 799: Project in Systems and Technology Part I
Master of Engineering in Systems and Technology

Faculty Lead: Dr. Moein Mehrtash
Course Instructor: Dr. Seshasai Srinivasan
Community Engagement Coordinators: Richard Allen & Salman Bawa
Community Partner Lead: Josh Lombardo Bottema
August 2024

Prepared by:
Matt Jing (400190117)
Sebastian Rivera (400091501)

# Table of Contents


Abstract ..... 4

Acknowledgements ..... 4

Project ..... 4

Objectives ..... 6

GNSS ..... 6

GNSS Accuracy and Real-Time Kinematic (RTK) Positioning ..... 6

Simultaneous Localization and Mapping (SLAM) ..... 7

Path-Planning Algorithms ..... 7

Oak D Camera ..... 8

Computer GPU Testing ..... 9

Enabling GPU Support ..... 12

Solution Methodology ..... 15

Key Nodes in the Package ..... 15

Published Topics ..... 16

Implementation on the Husky Robot ..... 16

Testing GNSS Module Positioning Accuracy ..... 17

Initial Testing with RViz ..... 17

Visualization on RViz ..... 17

GNSS Data Recording and Visualization on Google Maps ..... 18

GNSS Data Details ..... 18

Example of a sensor_msgs/NavSatFix Message ..... 19

Converting the Text File to KML ..... 20

Visualization on Google Maps ..... 20

Oak D Camera Testing ..... 21

Conclusions ..... 22

Future Work ..... 23

Transition to ROS 2 and Autoware Integration ..... 23

Real-World Testing and Validation ..... 23

Development of Path-Planning Algorithms ..... 23

Adtruck ced Visualization and Analysis ..... 24

Enhanced Drive-by-Wire Systems ..... 24

XYZ in combination with YOLOP ..... 24

Roboflow Trained Data Set with YOLOP ..... 24
Two Oak-D Cameras ..... 24
Messages to ROS for other sensor actuations ..... 24
Dashcam Capabilities ..... 25
References ..... 26

# Abstract

The GoWrench autonomous driving vehicle project, in collaboration with McMaster University for the past two years, aims to develop a high-level autonomous vehicle with off-road capabilities. Under the leadership of Dr. Moein Mehrtash, the project has formed specialized teams focusing on both the mechanical design and the development of autonomous driving functionalities, particularly using lidar and camera technologies.

Our team, specifically focused on autonomous driving functionalities, has concentrated efforts on integrating GNSS modules for navigation and high-resolution cameras for environmental sensing. The autonomous driving system is powered by an Nvidia Jetson Orin computer running the Ubuntu operating system. Key hardware components include the SwiftNav Duro GNSS module and the Luxonis Oak-D camera, both crucial for developing the vehicle's autonomous functions.

In the first term of the project, the team successfully addressed challenges related to ROS system integration. Significant achievements were made in verifying the accuracy of GNSS positioning data and implementing pre-trained object sensing algorithms.

# Acknowledgements

The progress made in the four months can be attributed to other previous taken courses in our masters university career, SEP 783 Sensors and Actuators, SEP 742 Visual Perception for Autonomous Vehicles, and other undergraduate capstones, projects and courses. An appropriate appreciation can also be made to Ping Li, Shiyu Chen and Jianfeng Zhang; these individuals were instrumental in testing the Clearpath Husky Robot with the GNNS and camera modules. And a final appreciation for the guidance of Dr. Moein Mehrtash, with his constant direction to challenge our approaches we were able to make great strides in getting started with these modules.

# Project Description

This project, in partnership with GoWrench Inc., aims to achieve autonomous driving functionality for a commercial truck provided by the company. At this initial stage of physical autonomous driving function integration, two key modules need to be incorporated into the computation system to fulfill basic autonomous driving requirements: the SwiftNav Duro GPS/GNSS package and the Luxonis Oak-D-Lite Camera package. These modules will be calibrated to work with the Nvidia Jetson Orin computation computer, and an autonomous driving algorithm will be developed and tested. Upon successful calibration and testing, the modules will be installed on the commercial truck from

GoWrench Inc. The goal is to prepare the truck for road testing by assembling these modules with the computation computer.

The GNSS module is essential for autonomous driving vehicles because it provides accurate and reliable positioning, which is crucial for determining the exact location on a map and navigating safely. GNSS allows vehicles to dynamically adjust their routes based on real-time traffic data, road conditions, and other factors, optimizing travel time and efficiency. One of the most common applications of the GNSS module in autonomous driving vehicles is path planning.

In this project, a SwiftNav Duro module is utilized to receive GNSS data, forming the foundation for developing adtruck ced path-planning algorithms. A parallel group is working on a similar project focusing on sidewalk robots, specifically using a Husky robot equipped with the necessary components for autonomous navigation, including a SwiftNav Duro GNSS receiver. Before testing the autonomous driving functionalities on a company vehicle intended for road use, we plan to test our algorithms on the Husky sidewalk robot. This approach offers a safer and more controlled environment for testing, as the robot moves much slower than a vehicle, allowing more time for analysis and adjustments. Additionally, the smaller size of the robot poses fewer risks during testing. Both the Husky robot and the target vehicle share the same GPU computer architecture, making it easier to transfer successful algorithms from the robot to the vehicle, thus saving considerable time and effort.

The SwiftNav Duro GNSS module can be connected to a GPU computer via either an Ethernet cable or a USB serial port. On the Husky sidewalk robot, the module is connected to a Jetson Orin computer through an Ethernet cable. Utilizing the available ROS package, ethz_piksi_multi, which is specifically tailored for integrating the SwiftNav Duro in robotic systems, we can export the GNSS data to develop and refine our path-planning algorithms. The original ROS launch file, piksi_multi_rover, is configured for USB serial connections, so modifications are necessary to adapt it for Ethernet-based implementations. Additionally, the ROS package includes a node that converts the output data from the ROS launch driver into KML format. This KML format can be directly imported into navigation applications like Google Maps for visualization. By combining the driver node with the KML conversion node, our customized ROS package can automate the collection and real-time conversion of GNSS data into KML for live position tracking. It's important to note that Swift Navigation provides a GUI console for GNSS data collection and signal quality visualization, but this console is only compatible with x86 Intel architecture. Since the Jetson Orin operates on Nvidia ARM architecture, using the ROS package is the most straightforward and effective method for collecting GNSS data on this platform.

The integration for the camera with depth sensing capabilities and IMU functionalities as well will provide intelligent autonomy, using pretrained models the GoWrench trucks will be able to navigate based on information received through various other sensors covered in this project in combination with others in development or already developed. YOLOP and YOLOV8 will be the main models that can provide object, road and vehicle detection.

*Figure 1. SwiftNav Duro GNNS receiver & Oak D Camera*

# Objectives

## GNSS

The primary function of the GNSS (Global Navigation Satellite System) module is to collect positioning data from available satellites. GNSS refers to a constellation of satellites that provide autonomous geo-spatial positioning with global coverage. The receiver determines the distance, or range, to each satellite by calculating the time difference between the broadcast time sent by the satellite and the time of signal reception. Upon receiving signals from multiple satellites, the GNSS device can ascertain the position of each satellite and its respective distance. Accurate positioning necessitates signals from a minimum of four satellites, due to the following reasons:

Single Satellite: The potential position is any point on a sphere with a radius equal to the distance from the satellite.
Two Satellites: The position is confined to the circular area where two spheres intersect.
Three Satellites: The position is further constrained to a line on the Earth's surface.
Four Satellites: The intersection of spheres provides a precise location.
The reception of signals from a greater number of satellites results in more intersections of these spheres, thereby enhancing positional accuracy. Several GNSS systems are currently in operation, including GPS (Global Positioning System) developed by the United States, GLONASS (Globalnaya Navigazionnaya Sputnikovaya Sistema) developed by Russia, Galileo from the European Union, BeiDou from China, and QZSS (Quasi-Zenith Satellite System), a regional GNSS operated by Japan.

### GNSS Accuracy and Real-Time Kinematic (RTK) Positioning

The standard accuracy of GNSS receivers ranges from 2 to 10 meters under outdoor conditions. To achieve centimeter-level accuracy, Real-time Kinematic (RTK) positioning technology can be employed. RTK positioning requires an additional fixed receiver, known as the base station, which remains stationary and transmits correction signals to the mobile receiver, referred to as the rover. Both the rover and the base station receive satellite signals, and the rover utilizes RTK correction signals to attain centimeter-level positioning accuracy. It is essential for both the base station and the rover to receive

signals from a similar set of satellites, thus making RTK corrections valid up to a distance of 50 km from the base station. For autonomous driving applications, the accuracy of positioning data is of paramount importance. However, utilizing RTK correction services involves substantial costs. Therefore, it is imperative to validate the accuracy of positioning data derived solely from GNSS signals before deciding on the necessity of employing RTK correction services.

### Integration with Autoware and Drive-by-Wire Systems

Navigation and path-planning are critical functions in autonomous driving, and they can be achieved through various methods. Autoware, an open-source project for autonomous driving built on the Robot Operating System (ROS), supports numerous autonomous driving functions, including localization, object detection, path planning, and control. Autoware can integrate high-precision GNSS data from the ethz_piksi_ros package to enhance vehicle localization. A drive-by-wire system includes modules for steer-by-wire and throttle-by-wire, allowing the vehicle to be controlled through software commands. Our project aims to utilize ethz_piksi_ros to integrate precise GNSS data for improved localization and navigation. The GNSS data will be used alongside other sensors, such as LiDAR and cameras, to provide a comprehensive view of the vehicle's environment.

## Simultaneous Localization and Mapping (SLAM)

Simultaneous Localization and Mapping (SLAM) algorithms enable a vehicle to map unknown environments while determining its location within that map. Despite the accuracy provided by SwiftNav GNSS data, integrating ethz_piksi_ros into a SLAM system within a ROS-based autonomous vehicle project will enhance localization and mapping performance. SLAM algorithms use information from wheel encoders, cameras, and other imaging sensors to determine movement and create maps of the surroundings, avoiding redundant coverage.

## Path-Planning Algorithms

Although the operating system is based on ROS 1, and Autoware is currently only supported by ROS 2, the path-planning functions will need to be developed from scratch at this stage of the project. Several popular path-planning algorithms can be integrated into a ROS-based system, including:

### A (A-star) Algorithm*:

Description: A popular pathfinding algorithm known for its efficiency and accuracy.
Application: Suitable for urban navigation where the environment can be discretized into a grid. High-precision GNSS data can determine the vehicle's position on this grid.

### Dijkstra's Algorithm:

Description: A well-known algorithm for finding the shortest paths between nodes in a graph.
Application: Effective in structured environments like city streets, where paths can be represented as a weighted graph.

### Rapidly-exploring Random Tree (RRT):

Description: Designed for efficiently searching non-convex, high-dimensional spaces by randomly building a space-filling tree.
Application: Suitable for off-road or semi-structured environments where obstacles are irregular. High-precision GNSS data is crucial for accurate localization.

### Probabilistic Roadmap (PRM):

Description: Suitable for complex environments with multiple potential routes.
Application: Effective for environments with numerous potential paths where planning needs to be flexible.

### Model Predictive Control (MPC):

Description: Suitable for high-speed driving and environments where the path needs constant adjustment based on real-time data.
Application: Utilizes high-precision GNSS data to continuously adjust the vehicle's path, making it ideal for dynamic driving conditions.

Incorporating high-precision GNSS data from ethz_piksi_ros into our autonomous driving project will significantly improve localization and navigation. By combining GNSS data with SLAM algorithms and integrating robust path-planning algorithms, a high level of accuracy and reliability in autonomous navigation can be achieved. This approach ensures the vehicle can navigate both structured urban environments and unstructured off-road scenarios effectively.

## Oak D Camera

The Oak D camera is a very capable camera that can be utilized with its on-board processing power of 1.4 TOPs for artificial intelligence algorithms or on-board GPU on the computer. Enabling the GPU on the camera will provide real time object identification or any other functions available via other libraries such as open computer vision and YOLO. Another advantage of this camera is that it contains two stereo cameras that can record the depth of objects.

## Computer GPU Testing

Extensive computer GPU testing was conducted on a Lenovo ThinkPad Ps14 Gen 4 with an RTX A500 graphics card. While the Jetson Orin boasts a computing power of 275 TOPS of AI performance which allows it to carry out real-image processing and other AI driver applications.

The appropriate software was enabled on a Ubuntu partition on a windows laptop and the testing was conducted to run YOLOV8.

YoloV8 was chosen as the starting point since it is a model that enables high speed and accuracy of object identification. The following commands needed to be taken for the GPU support the Oak D Camera;

Steps to get camera to work

1. Set up the Environment
   sudo apt update
   sudo apt upgrade
2. Install Python and Pip
   sudo apt install python3 python3-pip
3. Create and Activate a Virtual Environment
   sudo apt install python3-venv
   python3 -m venv yolov8-env
   source yolov8-env/bin/activate
4. Install DepthAI
   pip install depthai
5. Install OpenCV
   pip install opencv-python
6. Install YOLOv8
   pip install ultralytics
7. Download the YOLOv8 model
   wget https://github.com/ultralytics/assets/releases/download/v0.0.0/yolov8s.pt

The commands above were necessary for the GPU to be working in sync with the camera. The DepthAI library provides an API for interacting with Oak D cameras. Thes testing and results will be analyzed in the following sections of this report.

The code below is responsible for the XYZ calculations:

It checks for GPU, camera and continues if this hardware is connected and present.

```
import depthai as dai
import cv2
from ultralytics import YOLO
```

```
import torch
import numpy as np

# Check for CUDA availability
print("CUDA available:", torch.cuda.is_available())
print("CUDA device count:", torch.cuda.device_count())
if torch.cuda.is_available():
print("CUDA device name:", torch.cuda.get_device_name(0))
else:
print("CUDA is not available. Using CPU.")

# Initialize YOLOv8 model and set to use GPU if available
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model = YOLO('yolov8n.pt').to(device)

# Print a message to verify whether the model is using the CPU or GPU
if device == 'cuda':
print("Using GPU for inference")
else:
print("Using CPU for inference")

# Create DepthAI pipeline
pipeline = dai.Pipeline()

# Define the ColorCamera node
camRgb = pipeline.create(dai.node.ColorCamera)
camRgb.setPreviewSize(640, 480)
camRgb.setInterleaved(False)
camRgb.setColorOrder(dai.ColorCameraProperties.ColorOrder.BGR)

# Define the MonoCamera nodes for depth calculation
left = pipeline.create(dai.node.MonoCamera)
right = pipeline.create(dai.node.MonoCamera)
left.setBoardSocket(dai.CameraBoardSocket.CAM_B)
right.setBoardSocket(dai.CameraBoardSocket.CAM_C)
left.setResolution(dai.MonoCameraProperties.SensorResolution.THE_800_P)
right.setResolution(dai.MonoCameraProperties.SensorResolution.THE_800_P)

# Define the StereoDepth node
stereo = pipeline.create(dai.node.StereoDepth)
stereo.setConfidenceThreshold(200)
stereo.initialConfig.setMedianFilter(dai.StereoDepthProperties.MedianFilter.KERNEL_7x7)
left.out.link(stereo.left)
right.out.link(stereo.right)
```

```
# Create output nodes to stream the camera frames and depth data
xoutRgb = pipeline.create(dai.node.XLinkOut)
xoutRgb.setStreamName("rgb")
camRgb.preview.link(xoutRgb.input)

xoutDepth = pipeline.create(dai.node.XLinkOut)
xoutDepth.setStreamName("depth")
stereo.depth.link(xoutDepth.input)

# Baseline and focal length for depth calculation
baseline = 75.0 # mm
focal_length = 883.57 # in pixels, actual calibration data

# Connect to the device and start the pipeline
with dai.Device(pipeline) as device:
qRgb = device.getOutputQueue(name="rgb", maxSize=4, blocking=False)
qDepth = device.getOutputQueue(name="depth", maxSize=4, blocking=False)

while True:
inRgb = qRgb.get()
inDepth = qDepth.get()

frame = inRgb.getCvFrame()
depthFrame = inDepth.getFrame()

# Perform inference on the frame using YOLOv8
results = model(frame)

# Visualize the results on the frame
annotated_frame = results[0].plot()

# Calculate the XYZ coordinates for each detected object
for detection in results[0].boxes:
x1, y1, x2, y2 = map(int, detection.xyxy[0])
centerX, centerY = (x1 + x2) // 2, (y1 + y2) // 2

# Get depth value at the center of the bounding box
depthValue = depthFrame[centerY, centerX]
if depthValue != 0: # Make sure depth value is valid
# Calculate real-world coordinates
Z = depthValue / 10.0 # Convert from mm to cm
X = (centerX - frame.shape[1] / 2) * Z / focal_length
Y = (centerY - frame.shape[0] / 2) * Z / focal_length
```

```
# Draw the coordinates on the frame
annotated_frame = cv2.putText(annotated_frame, f"X: {X:.2f} Y: {Y:.2f} Z: {Z:.2f}",
(x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2)

# Display the frame
cv2.imshow("OAK-D Camera", annotated_frame)

# Exit the loop if 'q' is pressed
if cv2.waitKey(1) == ord('q'):
break

cv2.destroyAllWindows()
```

## Enabling GPU Support

To enable the GPU on the Jetson Orin there must be a docker, a docker allows the automation of deploying, scaling and managing of different applications in small light weight container. This means that the docker image created can be downloaded on another Jetson Orin and should be able to use all the dependencies, libraries and algorithms without issues, it is able to work in different environments. This allows the isolation of the docker image to be able to run without affecting other system dependencies, which when setting up a custom docker for this application is very important.

The summarized steps to enabling GPU support, two main things must be considered, the Jetpack SDK and the Docker.

Referencing the Nvidia developer website "**Getting Started with Jetson AGX Orin Developer Kit**"

## Step 2 - Install JetPack Components

Once the initial setup is complete, you can install the latest JetPack components that correspond to your L4T version from the Internet.

Open a terminal window if you are on Ubuntu desktop ( ^ Ctrl + ⌥ Alt + T ).

**Attention**

Check your L4T version first to see if you have a unit flashed with older version of the BSP.

```
cat /etc/nv_tegra_release
```

You may get something like this, `# R34 (release), REVISION: 1.0, GCID: 30102743, BOARD: t186ref, EABI: aarch64, DATE: Wed Apr 6 19:11:41 UTC 2022`, and this shows that you have L4T for JetPack 5.0 Developer Preview.

If you have an earlier version of L4T, issue the following command to manually put the apt repository entries using commands below.

```
sudo bash -c 'echo "deb https://repo.download.nvidia.com/jetson/common r34.1 main" >>
/etc/apt/sources.list.d/nvidia-l4t-apt-source.list'
sudo bash -c 'echo "deb https://repo.download.nvidia.com/jetson/t234 r34.1 main" >>
/etc/apt/sources.list.d/nvidia-l4t-apt-source.list'
```

If you see `R34 (release), REVISION: 1.0` or newer, then your apt sources lists are already up to date and you can proceed

Issue the following commands to install JetPack components.

```
sudo apt update
sudo apt dist-upgrade
sudo reboot
sudo apt install nvidia-jetpack
```

It can take about an hour to complete the installation (depending on the speed of your Internet connection).

*Figure 2. JetPack Install*

Running the commands above will allow a plethora of components that can aid in accelerating and supporting the GPU intensive processes, some of these tools include, CUDA, cuDNN, OpenCV and TensorRT.

**CUDA** – (compute unified device architecture) this library allows for the GPU to be used for general purposes, it is responsible for the parallel processing that accelerates matrix multiplication, image processing and deep learning training and inference. For this project this is critical for the image processing speeds and accuracy.

**CuDDN** – (CUDA deep neural network library) this allows for the utilization of highly optimized implementations of deep learning operations, like backpropagation, forward and backward convolution.

**OpenCV** – is an open-source computer vision library with GPU acceleration available

**TensorRT** – this would be a high-performance deep learning library; it optimizes trained models for future use.

Jetpack 5.1.2 was downloaded with the following packages, libraries and tools:

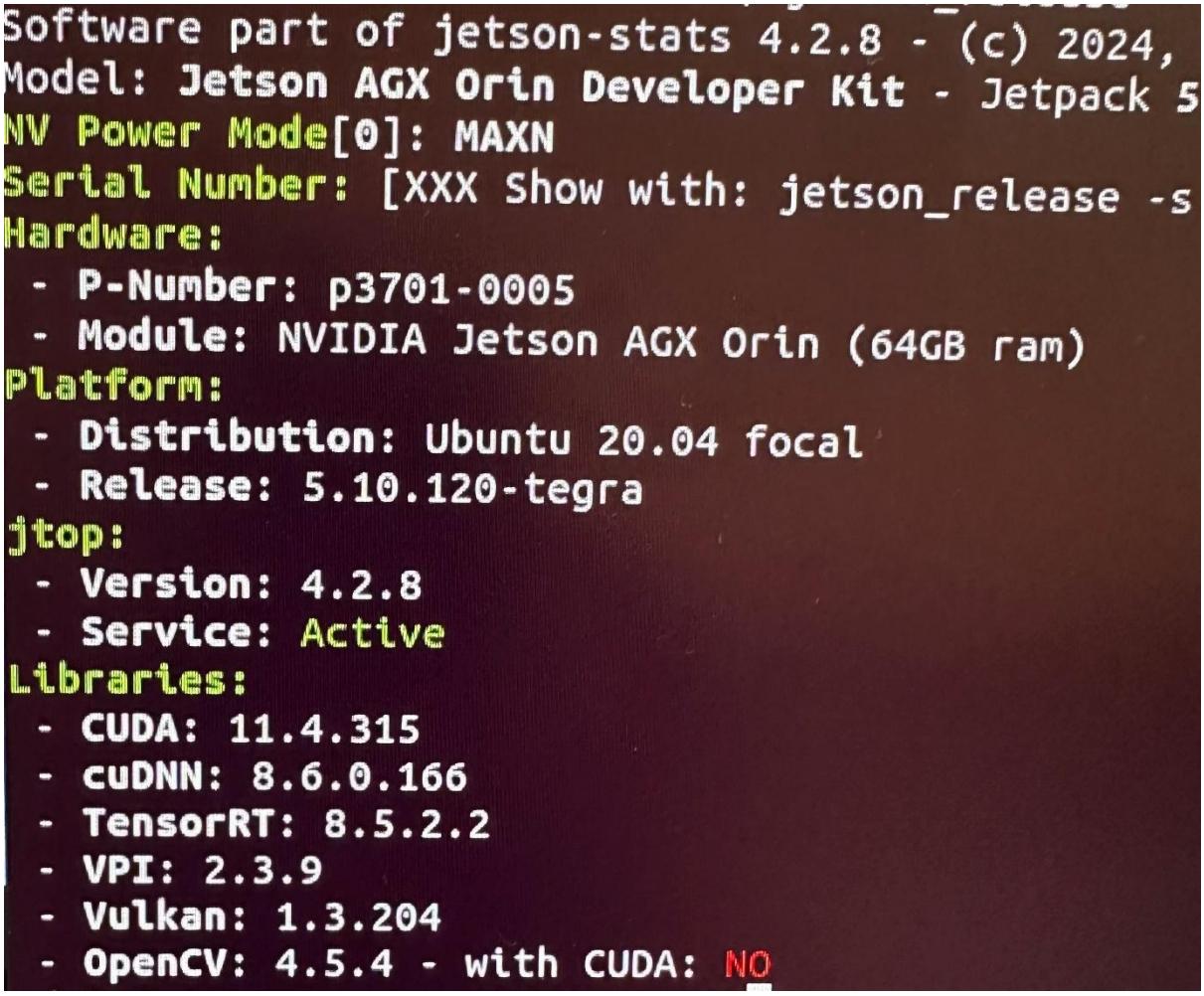



*Figure 3. Jetson Orin Software configuration*

The docker provides a way to keep the specific library and dependencies contained in an environment without affecting the overall Jetson Orin system configuration, the libraries and their respective version are above in the screenshot. The VPI (vision programming interface) library is an api for multiple backends like CUDA, this allows scripts on Nivida powered devices. Vulkan is also another api that utilizes 3D graphics and computational algorithms, this low-overhead api also can control GPU operations; allowing it to handle resource management properly and effectively.

# Solution Methodology

The ethz_piksi_ros package is crucial for integrating SwiftNav GNSS receivers into ROS-based systems, enabling efficient collection, processing, and utilization of GNSS data within the ROS ecosystem. This integration is pivotal for achieving high-precision localization and navigation in autonomous vehicle applications. Below are the primary functionalities of the package:

Interfacing with SwiftNav Piksi Multi:
The package provides comprehensive drivers and tools for seamless connection and communication with the Piksi Multi GNSS receiver.

Publishing GNSS Data:
It publishes vital GNSS data, such as position, velocity, and time, to ROS topics, making this data readily accessible to other ROS nodes and packages for enhanced system interoperability.

RTK Corrections:
The package supports receiving and applying Real-Time Kinematic (RTK) corrections, which significantly enhance the accuracy of GNSS positioning, crucial for precision-dependent applications.
Monitoring and Debugging:

It includes robust tools for monitoring the status and performance of the GNSS receiver, along with debugging capabilities to resolve operational issues efficiently.

## Key Nodes in the Package

The ethz_piksi_ros package contains several essential nodes, with the following being particularly critical for system communication and data collection:

piksi_multi_rtk_ros:
This ROS driver node for the Piksi RTK receiver manages communication with the Piksi Multi receiver, processes incoming data, and publishes it to ROS topics. It employs custom ROS message types to encapsulate various GNSS data and RTK correction information.
Pre-configured ROS launch files simplify node initialization and configuration setup, streamlining the deployment process.

piksi_rtk_kml:
This node supports the integration and utilization of GNSS data, although it is not as extensively detailed in this context.

## Published Topics

The piksi_multi_rtk_ros node publishes several critical topics, including:

/piksi/navsatfix_rtk_fix (sensor_msgs/NavSatFix): Contains WGS 84 coordinates when an RTK fix is achieved.
/piksi/navsatfix_spp (sensor_msgs/NavSatFix): Contains WGS 84 coordinates for SPP fix positions.
/piksi/navsatfix_best_fix (sensor_msgs/NavSatFix): Contains WGS 84 coordinates with the best available fix (RTK or SPP).
/piksi/debug/receiver_state (piksi_rtk_msgs/ReceiverState_V2_2_15): Provides miscellaneous information about the state of the Piksi Multi receiver.
/piksi/enu_pose_fix (geometry_msgs/PointStamped): Contains ENU (East-North-Up) coordinates of the receiver when an RTK fix is achieved, with orientation set to identity quaternion (w=1).
/piksi/enu_pose_spp (geometry_msgs/PointStamped): Contains ENU coordinates for SPP fix positions, with orientation set to identity quaternion (w=1).
/piksi/enu_pose_best_fix (geometry_msgs/PointStamped): Contains ENU coordinates with the best available fix (RTK or SPP), with orientation set to identity quaternion (w=1).

## Implementation on the Husky Robot

For practical implementation, the SwiftNav Duro module is connected to the Jetson Orin computer via an ethernet cable on the test-oriented Husky robot. The pre-configured launch file must be modified to activate the GNSS positioning tracking function. Key modifications include setting the tcp_addr to match the SwiftNav Duro IP address and setting the base_station_ip to the robot's IP address for network status verification. Once the launch file is executed, the device operates in rover mode, with GNSS data published under the piksi topic.

This integration ensures that high-precision GNSS data is effectively utilized within the ROS-based autonomous vehicle project, enhancing localization, navigation, and overall system performance. By leveraging the capabilities of the ethz_piksi_ros package, the project can achieve robust and accurate positioning, crucial for adtruck ced autonomous driving functionalities.

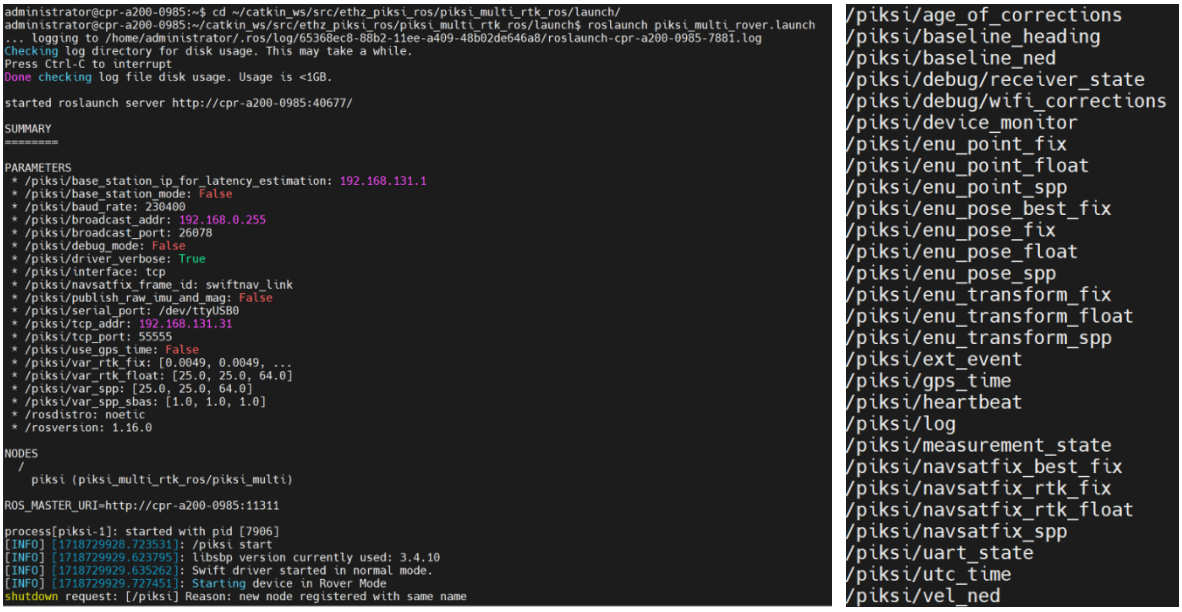



*Fig. 2 &3. SwiftNav Duro GNSS receiver driver file output and published GNSS topics*

## Testing GNSS Module Positioning Accuracy

After confirming communication between the GNSS module and the system, and successfully obtaining GNSS data topics, the robot is tested outdoors for positioning accuracy. This is necessary because the device cannot receive GNSS signals inside a building, resulting in no data output.

## Initial Testing with RViz

To visualize the GNSS data instantly, the RViz visualization tool is used, which requires installing an additional plugin—RViz Satellite. This plugin abstracts the messages contained in the /piksi/navsatfix_best_fix topic and displays the position on a map from a specified URL link. While this method provides a rapid solution for verifying positioning data, it offers only a rough accuracy check because it cannot label all the steps recorded in the GNSS topic.

## Visualization on RViz

*Fig. 4* illustrates the testing area, providing a preliminary indication that the GNSS data accuracy is acceptable. However, for a more thorough verification, the messages contained in the GNSS topics need to be recorded and imported into advantage visualization tools like Google Maps or Google Earth Pro. These tools allow for detailed analysis and verification of the positioning data, ensuring a higher level of accuracy assessment.

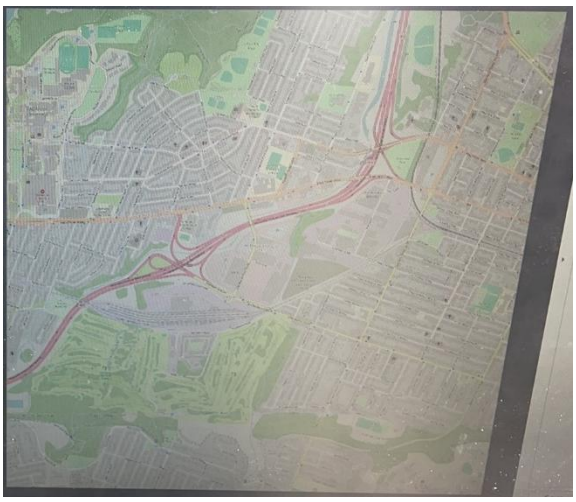


*Fig. 4 GNSS visualization on Rviz satellite*

## GNSS Data Recording and Visualization on Google Maps

To visualize positioning data on Google Maps, it is necessary to convert the data into a KML (Keyhole Markup Language) file. The ROS system cannot directly save data in KML format, but there are multiple methods to record messages from the /piksi/navsatfix_best_fix topic, including using rosbag, recording messages in a .txt file, or using a node from the ethz_piksi_ros package.

Recording Methods
Using rosbag:
Record the data into a .bag file, which can be processed later to extract and convert the data.
Direct Recording:
Record the echoed messages from the topic into a .txt file. This is the most straightforward method and involves minimal setup.
Package Node:
Use specific nodes within the ethz_piksi_ros package designed for data recording.
For simplicity, we will focus on the method of directly recording messages in a .txt file and converting this file into a KML format using a Python script.

## GNSS Data Details

The message in the /piksi/navsatfix_best_fix topic, of type sensor_msgs/NavSatFix, contains several pieces of information about the GNSS position. The main components of the sensor_msgs/NavSatFix message include:
Header:
seq: Sequential ID, a unique identifier for the message.
stamp: ROS time stamp, indicating the time when the data was received.
frame_id: Frame of reference for the data (e.g., "gps").

Status:
status: Fix status indicator, with possible values:
-1: No fix
0: Unaugmented fix (standard GPS)
1: With differential corrections (DGPS)
2: RTK float solution
3: RTK fixed solution
service: GNSS service used (e.g., GPS, GLONASS, etc.).

Latitude: Latitude in degrees.

Longitude: Longitude in degrees.

Altitude: Altitude in meters above the WGS 84 ellipsoid.

Position Covariance:
position_covariance: An array of nine elements representing the covariance matrix of the position estimate.
position_covariance_type: Type of the covariance matrix, indicating the method used to compute the covariance:
0: Unknown
1: Approximated
2: Diagonal known
3: Known

## Example of a sensor_msgs/NavSatFix Message

*Fig.5* illustrates a typical sensor_msgs/NavSatFix message:

```
header:
  seq: 42
  stamp:
    secs: 1623185096
    nsecs: 987654321
  frame_id: "gps"
status:
  status: 2
  service: 1
latitude: 37.4219999
longitude: -122.0840575
altitude: 5.0
position_covariance: [0.1, 0, 0, 0, 0.1, 0, 0, 0, 0.1]
position_covariance_type: 2
```

*Fig. 5 Example of a sensor_msgs/NavSatFix message*

## Converting the Text File to KML

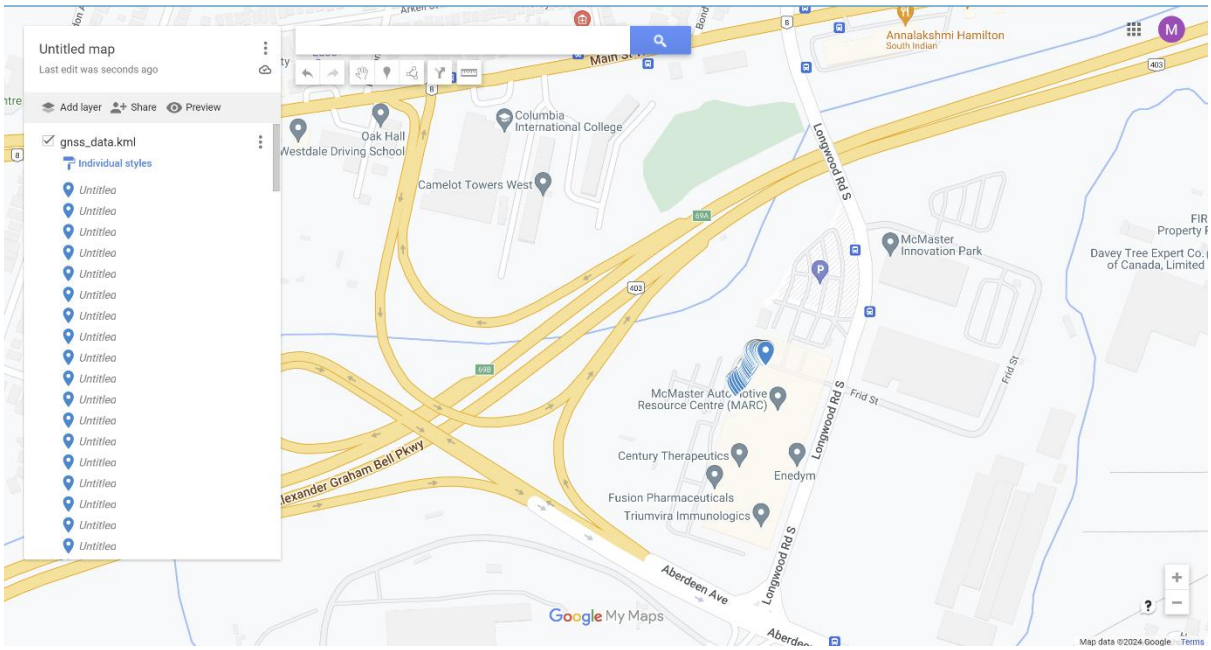
After recording the data in a .txt file, it can be converted to KML format using a Python script. Here is an example script for the conversion:

```
import pandas as pd
import simplekml

# Load the text file into a DataFrame
txt_file = 'piksi_navsatfix_best_fix.txt'
df = pd.read_csv(txt_file, delim_whitespace=True)

# Create a KML object
kml = simplekml.Kml()

# Loop through the DataFrame and add points to the KML
for index, row in df.iterrows():
    kml.newpoint(name=f"Point {index}", coords=[(row['longitude'], row['latitude'])])

# Save the KML file
kml.save("gnss_data.kml")
```

## Visualization on Google Maps

After converting the .txt file into a KML format file, the data can be visualized on Google Maps or Google Earth. As illustrated in *Fig. 6*, the positioning data is reasonably accurate even without using the RTK correction service, demonstrating the effectiveness of the GNSS module in providing reliable location data.

By recording and converting GNSS data into a KML file, we can leverage advantage visualization tools to analyze and verify the accuracy of the positioning data, ensuring the robustness of the autonomous vehicle's navigation system.




# Oak D Camera Testing

The following results of the code running are as follows:

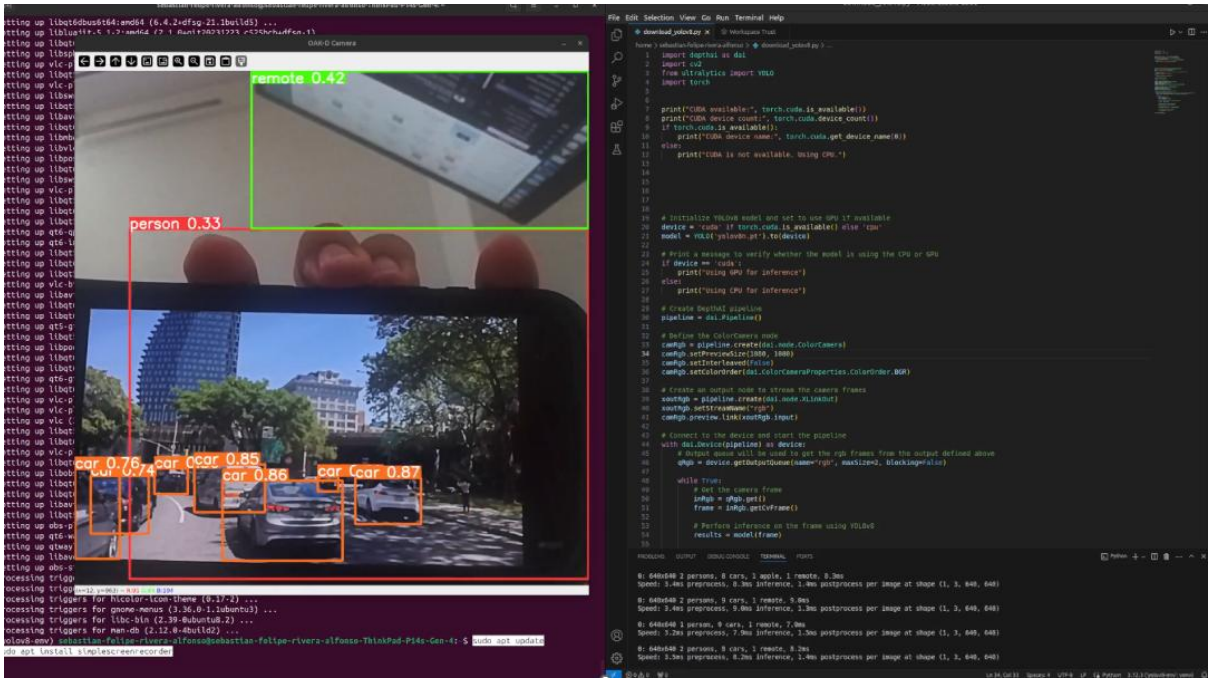



*Figure 4.Computer GPU YOLOV8*

The YOLOV8 model was able to take advantage of the GPU and perform very well with the videos it was put under. The “remote” had a confidence level of 0.42 which is accurate since it was a computer monitor while the fingers holding the phone presented a challenge for the model to decipher the accuracy if whether the fingers were part of a hand. While the cars were recognized at

confidence levels of 0.85. This simple test used the RBG camera of the oak d camera.

While the YOLOP running on the Jetson Orin is below. The road and car lane can be clearly differentiated, due to robot testing constraints the outdoor testing could not be completed, in the following semester, it will be done.

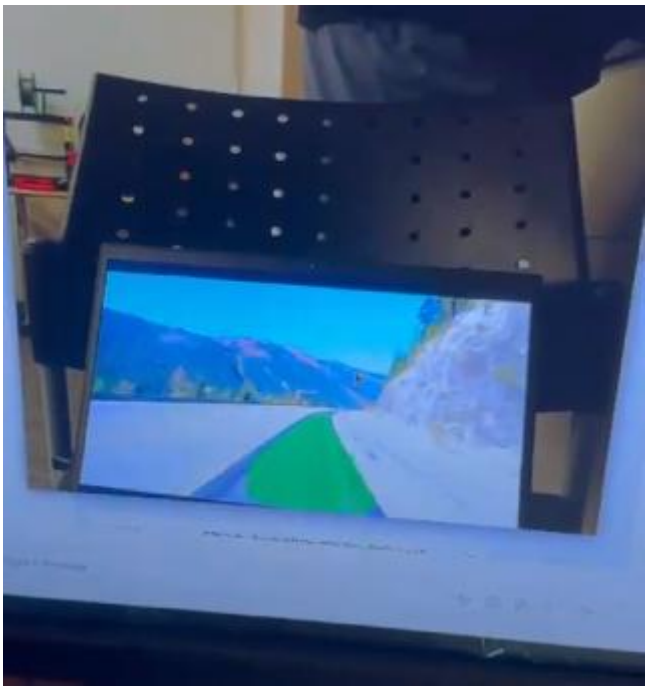


*Figure 5. Jetson Orin YOLOP*

# Conclusions

The GoWrench Inc. Autonomous Driving Vehicle Project, in collaboration with McMaster University, has made significant strides in developing an autonomous vehicle with off-road capabilities. The project focused on integrating GNSS modules for navigation and high-resolution cameras for environmental sensing, utilizing an Nvidia Jetson Orin computer. The primary GNSS module, the SwiftNav Duro, provided essential positioning data, crucial for path-planning and vehicle localization.

The team successfully integrated the ethz_piksi_ros package, which facilitated communication between the GNSS receiver and the ROS-based system. This integration enabled the collection, processing, and utilization of high-precision GNSS data. Various methods, including using RViz for initial visualization and converting GNSS data to KML format for detailed analysis in Google Maps, were employed to verify the accuracy of the positioning data. This thorough verification process confirmed that the GNSS data was reliable and suitable for autonomous navigation.

Recording GNSS data was achieved using various methods, including rosbag and direct recording into a .txt file. This data was then converted into KML format using Python scripts, enabling detailed

visualization and analysis in adtruck ced mapping tools. Initial visualization was done using RViz with the RViz Satellite plugin. For more detailed verification, the recorded data was visualized on Google Maps and Google Earth Pro, providing a comprehensive assessment of the GNSS data accuracy.

The SwiftNav Duro module was connected to the Jetson Orin computer via Ethernet on the Husky robot. The system was calibrated to ensure accurate positioning and efficient data transmission, setting the foundation for robust path-planning and navigation.

With the GNSS and camera module operating in harmony the navigation of the GoWrench trucks will be able to improve and significant cost savings and other benefits will incur.

# Future Work

Moving forward, the project will focus on several key areas to enhance the autonomous driving system's capabilities and reliability.

## Transition to ROS 2 and Autoware Integration

Transitioning from ROS 1 to ROS 2 will be a priority to leverage Autoware’s adtruck ced capabilities, particularly in path-planning and vehicle control. ROS 2 offers improved performance, better support for real-time systems, and enhanced security features, making it a more suitable platform for developing autonomous driving functionalities.

## Real-World Testing and Validation

The developed solutions will be implemented on the commercial vehicle for extensive real-world testing. This phase will involve rigorous testing under various conditions to ensure the system's reliability and robustness. The insights gained from these tests will be used to refine and improve the autonomous driving algorithms.

## Development of Path-Planning Algorithms

The team will develop and refine path-planning algorithms to enhance the vehicle's ability to navigate complex environments. Algorithms such as A*, Dijkstra’s, Rapidly-exploring Random Tree (RRT), Probabilistic Roadmap (PRM), and Model Predictive Control (MPC) will be considered. These algorithms will be tailored to different environments and requirements, enhancing the vehicle's ability to navigate both urban and off-road scenarios effectively.

## Adtruck ced Visualization and Analysis

To ensure the highest level of accuracy in positioning data, adtruck ced visualization tools will be used for detailed analysis. Converting GNSS data into KML format and visualizing it on platforms like Google Maps and Google Earth Pro will continue to be a critical part of the validation process. This approach will provide a comprehensive assessment of the positioning data's accuracy and reliability.

## Enhanced Drive-by-Wire Systems

Improving the drive-by-wire systems, which include modules for steer-by-wire and throttle-by-wire, will be essential. These systems will allow the vehicle to be controlled more precisely through software commands, enhancing the overall control and responsiveness of the autonomous vehicle.

## XYZ in combination with YOLOP

Utilizing the RBG and two grayscale stereo cameras with a script that invokes the YOLOP model plus a OpenCV function to measure the distance through depthAI's functionality this will improve and allow more accurate commands to ROS.

## Roboflow Trained Data Set with YOLOP

Roboflow allows the preparation of datasets for computer vision projects. Annotations and segmentation are all possible on Roboflow and they can be split in training, validation and testing datasets. This dataset can be images from the GoWrench facilities and possible environments where GoWrench sprinter truck s find themselves on work sites.

## Two Oak-D Cameras

Most automotive manufacturers are including more than one camera on their offering to be capable of a degree of autonomous driving. Implementing two will allow front and back surveillance and autonomous functionality to the truck s.

## Messages to ROS for other sensor actuations

Environment awareness is a large part of autonomous driving capabilities; the homologation and integration of these systems with ROS to output actual commands that can prompt an action is necessary.

## Dashcam Capabilities

Since the trucks will have internet connection capabilities, the cameras can capture video data for future training and insurance purposes. These cameras can be triggered by the IMU or other vehicle sensors to begin recording once a student stop or slow down is detected and with AI capabilities this could be further developed to recognize and verify an accident via the cameras and custom computer vision functions.

By focusing on these future objectives, the GoWrench autonomous driving vehicle project aims to develop a fully autonomous commercial vehicle with robust navigation and path-planning capabilities. These truck cements will ensure the vehicle can navigate both structured urban environments and unstructured off-road scenarios effectively, paving the way for safe and reliable autonomous driving solutions.

# References

https://github.com/ethz-asl/ethz_piksi_ros
https://support.swiftnav.com/support/solutions/articles/44001850760-duro-specification-and-user-manual
https://developer.nvidia.com/embedded/learn/get-started-jetson-agx-orin-devkit#step-2-install-jetpack-components
https://www.gowrenchauto.com/about
https://shop.luxonis.com/products/oak-d